

Aarya Arban

T11 – 05

Assignment No. 1

Aim: Study and implement basic Dart programs

Dart's Preliminary Concepts

1. Program Structure and Execution in Dart

Dart is a client-optimized programming language primarily used for building web, mobile, and desktop applications. Its simple yet robust structure allows developers to write clean and maintainable code. Below are the key aspects of Dart's program structure and execution:

Program Structure

1. Main Function:

- Every Dart program starts with the `main()` function.
- It is the entry point of the program and executes the code within it.

```
void main() {  
    print("Hello, Dart!");  
}
```

2. Libraries and Imports:

- Dart supports importing libraries for code reuse and modularity.
- Commonly used libraries include `dart:io`, `dart:math`, and `dart:core`.

Dart

```
import 'dart:math';
```

3. Statements and Expressions:

- Dart uses statements and expressions to perform operations.
- Semicolons (`;`) terminate statements.

4. Comments:

- Single-line comments: //
- Multi-line comments: /* ... */
- Documentation comments: ///

Execution in Dart

- Dart code is compiled into native machine code or JavaScript, depending on the target platform.
- It can run in two modes:
 1. **JIT (Just-in-Time) Compilation** during development for hot reloads.
 2. **AOT (Ahead-of-Time) Compilation** for optimized release builds.

2. Data Types in Dart

Dart is a statically typed language, meaning variables are assigned a type at compile-time. It also offers dynamic typing when required. Below are the commonly used data types:

Primitive Data Types

1. Numbers:

- int: Represents integer values.
- double: Represents decimal (floating-point) values.

```
int age = 25;
```

```
double price = 99.99;
```

1. Strings:

- Represents a sequence of characters enclosed in single or double quotes.

```
String name = "Dart Language";
```

2. Booleans:

- Represents true or false values.

```
bool isActive = true;
```

3. Lists:

- An ordered collection of items, similar to arrays.

```
List<int> numbers = [1, 2, 3];
```

4. **Maps:**

- Represents key-value pairs (explained below in detail).

Special Types

1. **Null:**

- By default, Dart supports null safety to avoid null-pointer exceptions.

2. **Dynamic:**

- A special type that allows variables to hold values of any type.

```
dynamic variable = "Hello";
```

```
variable = 42; // Valid
```

3. **Runes and Symbols:**

- Used for handling special Unicode characters and metadata.

3. Map Construct and Functions

Map Construct

A Map in Dart is a collection of key-value pairs, where each key is unique, and values can be repeated. Maps are highly useful for scenarios like dictionaries and configurations.

Key Features:

1. **Creating Maps:**

- Using literals or the Map class.

```
Map<String, int> phoneBook = {  
  'Alice': 12345,  
  'Bob': 67890,  
};
```

2. **Adding and Removing Elements:**

```
phoneBook['Charlie'] = 11223; // Add  
phoneBook.remove('Bob'); // Remove
```

3. Accessing Elements:

```
int? aliceNumber = phoneBook['Alice'];
```

4. Iterating through Maps:

```
phoneBook.forEach((key, value) {  
  print('$key: $value');  
});
```

Functions in Dart

Functions in Dart are first-class objects, meaning they can be assigned to variables, passed as arguments, and returned from other functions.

Types of Functions:

1. Named Functions:

- Declared with a name and invoked directly.

```
void greet(String name) {  
  print("Hello, $name!");  
}
```

2. Anonymous Functions:

- Functions without a name, typically used as inline expressions.

```
List<int> numbers = [1, 2, 3];  
numbers.forEach((num) {  
  print(num);  
});
```

3. Arrow Functions:

- A shorthand for single-expression functions.

```
int square(int num) => num * num;
```

4. Optional Parameters:

- Dart allows both positional and named optional parameters.

```
void describe(String name, {int? age}) {  
  print("Name: $name, Age: ${age ?? 'Unknown'}");  
}
```

Return Values:

- Functions can return values using the return keyword or simply evaluate an expression for arrow functions.

```
int add(int a, int b) {  
  return a + b;  
}
```

Exercise 1: Write a program in Dart that reads a string from the user and prints the following using functions:

1. Number of characters, numbers, and special characters

Program:

```
import 'dart:io';  
  
void main() {  
  print("Enter a string:");  
  String input = stdin.readLineSync()!;  
  
  print("\n Count of characters, numbers, and special characters:");  
  countCharacters(input);  
}  
  
void countCharacters(String input) {  
  int letters = 0, numbers = 0, specials = 0;
```

```

for (var char in input.runes) {
  if (RegExp(r'[a-zA-Z]').hasMatch(String.fromCharCode(char))) {
    letters++;
  } else if (RegExp(r'[0-9]').hasMatch(String.fromCharCode(char))) {
    numbers++;
  } else {
    specials++;
  }
}

print("Letters: $letters, Numbers: $numbers, Special Characters:
$specials");
}

```

Output:

Enter a string:

Aarya_15

Count of characters, numbers, and special characters:

Letters: 5, Numbers: 2, Special Characters: 1

2. Where the given string is palindrome or not

Program:

```

import 'dart:io';

void main() {
  print("Enter a string:");
  String input = stdin.readLineSync()!;

  print("\n2. Check if the string is palindrome or not:");
  checkPalindrome(input);
}

```

```

}

void checkPalindrome(String input) {
    String reversed = input.split("").reversed.join("");
    if (input == reversed) {
        print("$input is a palindrome.");
    } else {
        print("$input is not a palindrome.");
    }
}

```

Output:

Check if the string is palindrome or not:

radar is a palindrome.

3. Reverse of the string

Program:

```

import 'dart:io';

void main() {
    print("Enter a string:");
    String input = stdin.readLineSync()!;
    print("\n Reverse the string:");
    reverseString(input);
}

void reverseString(String input) {
    String reversed = input.split("").reversed.join("");

```

```
    print("Reversed String: $reversed");  
}
```

Output:

Enter a string:

bowler

Reverse the string:

Reversed String: relwob

Exercise 2: Write a program in Dart to create a contact dictionary with names, mobile numbers, and roll no of your friends, the program should be able to add, delete, display, and update the mobile numbers and names using the roll no.

Program:

```
import 'dart:io';  
  
void main() {  
    Map<int, Map<String, String>> contacts = {};  
  
    while (true) {  
        print("\n--- Contact Dictionary ---");  
        print("1. Add Contact");  
        print("2. Delete Contact");  
        print("3. Display Contacts");  
        print("4. Update Contact");  
        print("5. Exit");  
        stdout.write("Choose an option: ");  
        int choice = int.parse(stdin.readLineSync()!);
```



```
switch (choice) {  
    case 1:  
        addContact(contacts);  
        break;  
    case 2:  
        deleteContact(contacts);  
        break;  
    case 3:  
        displayContacts(contacts);  
        break;  
    case 4:  
        updateContact(contacts);  
        break;  
    case 5:  
        print("Exiting the program. Goodbye!");  
        return;  
    default:  
        print("Invalid choice. Please try again.");  
}  
}  
}
```

```
void addContact(Map<int, Map<String, String>> contacts) {  
    stdout.write("Enter roll number: ");  
    int rollNo = int.parse(stdin.readLineSync()!);
```

```
if (contacts.containsKey(rollNo)) {  
    print("Contact with roll number $rollNo already exists!");  
    return;  
}
```

```
stdout.write("Enter name: ");  
String name = stdin.readLineSync()!;  
stdout.write("Enter mobile number: ");  
String mobile = stdin.readLineSync()!;
```

```
contacts[rollNo] = {'name': name, 'mobile': mobile};  
print("Contact added successfully.");  
}
```

```
void deleteContact(Map<int, Map<String, String>> contacts) {  
    stdout.write("Enter roll number to delete: ");  
    int rollNo = int.parse(stdin.readLineSync()!);
```

```
    if (contacts.containsKey(rollNo)) {  
        contacts.remove(rollNo);  
        print("Contact deleted successfully.");  
    } else {  
        print("Contact with roll number $rollNo not found.");  
    }  
}
```

```
void displayContacts(Map<int, Map<String, String>> contacts) {  
    if (contacts.isEmpty) {  
        print("No contacts available.");  
        return;  
    }
```

```
    print("\nContacts List:");  
    contacts.forEach((rollNo, details) {  
        print("Roll No: $rollNo, Name: ${details['name']}, Mobile:  
        ${details['mobile']}");  
    });  
}
```

```
void updateContact(Map<int, Map<String, String>> contacts) {  
    stdout.write("Enter roll number to update: ");  
    int rollNo = int.parse(stdin.readLineSync());  
  
    if (!contacts.containsKey(rollNo)) {  
        print("Contact with roll number $rollNo not found.");  
        return;  
    }
```

```
    stdout.write("Enter new name: ");  
    String name = stdin.readLineSync();  
    stdout.write("Enter new mobile number: ");  
    String mobile = stdin.readLineSync();
```

```
contacts[rollNo] = {'name': name, 'mobile': mobile};  
print("Contact updated successfully.");  
}
```

Output:

--- Contact Dictionary ---

1. Add Contact
2. Delete Contact
3. Display Contacts
4. Update Contact
5. Exit

Choose an option: 1

Enter roll number: 5

Enter name: Aarya

Enter mobile number: 9757126166

Contact added successfully.

--- Contact Dictionary ---

1. Add Contact
2. Delete Contact
3. Display Contacts
4. Update Contact
5. Exit

Choose an option: 3

Contacts List:

Roll No: 5, Name: Aarya, Mobile: 9757126166

--- Contact Dictionary ---

1. Add Contact
2. Delete Contact
3. Display Contacts
4. Update Contact
5. Exit

Choose an option: 4

Enter roll number to update: 5

Enter new name: Aarya

Enter new mobile number: 9757126171

Contact updated successfully.

--- Contact Dictionary ---

1. Add Contact
2. Delete Contact
3. Display Contacts
4. Update Contact
5. Exit

Choose an option: 5

Exiting the program. Goodbye!

Exercise 3: Write a program in Dart to generate a random password using the following conditions from the user. the program should ask the following

1. Length of the password
2. whether the password is alphanumeric or only characters

3. what special characters should be included?

4. Whether uppercase characters allowed?

Program:

```
import 'dart:io';
import 'dart:math';

void main() {
  print("Random Password Generator");

  stdout.write("Enter the length of the password: ");
  int length = int.parse(stdin.readLineSync()!);

  stdout.write("Should the password be alphanumeric? (yes/no): ");
  bool isAlphanumeric = stdin.readLineSync()!.toLowerCase() == 'yes';

  stdout.write("Enter the special characters to include (leave blank for none): ");
  String specialChars = stdin.readLineSync()!;

  stdout.write("Allow uppercase characters? (yes/no): ");
  bool allowUppercase = stdin.readLineSync()!.toLowerCase() == 'yes';

  String password = generatePassword(length, isAlphanumeric, specialChars, allowUppercase);
  print("Generated Password: $password");
}
```

```
String generatePassword(int length, bool isAlphanumeric, String  
specialChars, bool allowUppercase) {
```

```
    String lowerCaseLetters = "abcdefghijklmnopqrstuvwxyz";
```

```
    String numbers = "0123456789";
```

```
    String allowedChars = lowerCaseLetters;
```

```
    if (allowUppercase) {
```

```
        allowedChars += lowerCaseLetters.toUpperCase();
```

```
    }
```

```
    if (isAlphanumeric) {
```

```
        allowedChars += numbers;
```

```
    }
```

```
    allowedChars += specialChars;
```

```
    Random random = Random();
```

```
    String password = "";
```

```
    for (int i = 0; i < length; i++) {
```

```
        int index = random.nextInt(allowedChars.length);
```

```
        password += allowedChars[index];
```

```
    }
```

```
    return password;
```

```
}
```

Output:

Random Password Generator

Enter the length of the password: 7

Should the password be alphanumeric? (yes/no): yes

Enter the special characters to include (leave blank for none): #_

Allow uppercase characters? (yes/no): yes

Generated Password: UxMds8c